Integración por Monte Carlo en memoria compartida con OpenMP

Evaluación de programación paralela

Universidad Industrial de Santander • 28661 • Docente: Sergio Augusto Gélvez Cortés

Sergio Andrés Pinzón Castellanos - 2202006

Introducción a la Programación Paralela

Universidad Industrial de Santander

2026-2

Instrucciones generales

1. Lea completamente la evaluación antes de comenzar a escribir o a programar. Las cuatro partes están relacionadas: el trabajo experimental de las partes II y III alimenta el informe de la parte IV. habrá una **sustentación oral breve** sobre su propio trabajo.
2. Responda con rigor y en español. Se califica el proceso de razonamiento y la justificación de sus decisiones, no únicamente el resultado numérico. Toda respuesta sin justificación se considera incompleta.
3. Puede consultar el material del curso, la documentación oficial de OpenMP y del compilador GCC, y puede apoyarse en herramientas de inteligencia artificial generativa, sujetas a dos condiciones: (i) debe declarar todo su uso en la *bitácora de IA* que acompaña a su informe (herramienta consultada, propósito, qué aceptó, qué descartó y por qué), y (ii) debe estar en capacidad de explicar y defender todo el contenido de su entrega, incluido el que haya elaborado con ayuda de esas herramientas. La ausencia de bitácora en un trabajo que la requiera se trata como falta de academicidad.
4. Ponderación de la evaluación: Parte I, 15 %; Parte II, 35 %; Parte III, 30 %; Parte IV, 20 %. La parte opcional otorga puntos adicionales según el criterio del docente.

Parte I. Fundamento del método (15 %)

Antes de comenzar, lea con atención la siguiente síntesis del método que usará durante toda la evaluación.

La integración por Monte Carlo en pocas palabras

La integración por Monte Carlo estima el valor de una integral mediante muestreo aleatorio. Para aproximar

$$I = \int_a^b f(x)\,dx,$$

se generan N puntos $x_1, x_2, \ldots, x_N$ distribuidos de manera uniforme en $[a, b]$ y se calcula el estimador

$$I_N = \frac{b-a}{N} \sum_{i=1}^{N} f(x_i).$$

Por la ley de los grandes números, I_N converge al valor exacto de la integral cuando N crece; el error estadístico típico disminuye en proporción a $1/\sqrt{N}$, de modo que cuadruplicar el número de muestras reduce el error a la mitad.

Un caso particular le servirá en esta evaluación: el área del cuarto de círculo unitario es $\pi/4$, es decir,

$$\int_0^1 \sqrt{1-x^2}\,dx = \frac{\pi}{4}.$$

En la variante de *acierto y fallo*, se generan puntos (x, y) uniformes en el cuadrado $[0, 1] \times [0, 1]$ y se cuenta la fracción que cae dentro del cuarto de círculo $x^2 + y^2 \le 1$; esa fracción multiplicada por 4 aproxima π. Cada muestra es independiente de las demás, y lo único que se acumula a lo largo del cálculo es el número de aciertos.

Entorno de trabajo. Todo lo hice en Ubuntu sobre WSL (Windows Subsystem for Linux), con GCC y OpenMP, en un portátil con procesador Intel Core i5-1235U (12.ª generación); según lscpu, WSL ve 12 CPU lógicas, L2 de 7,5 MiB y L3 de 12 MiB. Aunque la Parte I es teórica, quise comprobar con un experimento corto lo que afirmo en I.1, así que escribí un programa pequeño (error_vs_n.c) que repite la estimación de π con 20 semillas distintas para varios N y calcula el error cuadrático medio. Lo compilé y ejecuté así:

```
sudo apt install build-essential      # gcc, make y libgomp (OpenMP)
gcc -O2 error_vs_n.c -o error_vs_n -lm
./error_vs_n
```

I.1. ¿Por qué el error baja como 1/√N y qué cuesta ganar precisión?

La idea que me ayudó a entenderlo es que el estimador es un promedio de N variables aleatorias independientes. En la variante de acierto y fallo, cada punto produce un valor X_i que vale 1 si cae dentro del cuarto de círculo y 0 si no; es decir, una Bernoulli con probabilidad $q = \pi/4$. El estimador es

$$\pi\hat{N} = 4 \cdot (1/N) \cdot \Sigma X_i , \quad E[\pi\hat{N}] = 4q = \pi$$

Como las muestras son independientes, las varianzas se suman: la varianza de la suma es $N \cdot q(1-q)$, y al dividir entre N (y multiplicar por 4) queda

$$\mathrm{Var}(\pi\hat{N}) = 16 \cdot q(1-q) / N \quad \Rightarrow \quad \sigma(\pi\hat{N}) = 4 \cdot \sqrt{[q(1-q)]} / \sqrt{N} \approx 1{,}642 / \sqrt{N}$$

Lo importante es que la varianza baja como 1/N, pero el error que uno observa se mide en las mismas unidades que π, o sea con la desviación estándar, que es la raíz de la varianza. Por eso el error baja como $1/\sqrt{N}$ y no como 1/N. Intuitivamente: los errores de cada muestra van en direcciones aleatorias y se cancelan parcialmente, pero no del todo. Lo mismo pasa con la fórmula general $(b-a)/N \cdot \Sigma f(x_i)$: su desviación es $(b-a) \cdot \sigma f/\sqrt{N}$.

Reducir el error a la mitad: si quiero $\sigma(N') = \sigma(N)/2$, necesito $1/\sqrt{N'} = 1/(2\sqrt{N})$, o sea $N' = 4N$. Es decir, necesito 3N muestras adicionales (cuadruplicar). En general, dividir el error por k exige multiplicar N por k^2.

Costo de un dígito: ganar un dígito significa dividir el error entre 10, lo que exige $10^2 = 100$ veces más muestras (99N adicionales) y, como el tiempo es proporcional a N, unas 100 veces más tiempo. Con $N = 10^8$ el error esperado es $\approx 1{,}6\times10^{-4}$ (unos 3–4 decimales correctos); para tener 6 decimales necesitaría del orden de 10^{12} muestras. Esto me dejó claro que Monte Carlo converge lento y que paralelizar sirve para hacer más muestras en el mismo tiempo, no para cambiar esa tasa.

Al repetir la ejecución obtuve exactamente los mismos valores, porque rand_r con la misma semilla genera la misma secuencia. Mi experimento coincide con la teoría: al multiplicar N por 4 el error se reduce aproximadamente a la mitad, y por 100 se reduce unas 10 veces.

N	Error RMS medido (20 semillas)	Teórico $1{,}642/\sqrt{N}$
10^4	$1{,}82\times10^{-2}$	$1{,}64\times10^{-2}$
4×10^4	$7{,}52\times10^{-3}$	$8{,}21\times10^{-3}$
10^6	$1{,}53\times10^{-3}$	$1{,}64\times10^{-3}$
4×10^6	$7{,}57\times10^{-4}$	$8{,}21\times10^{-4}$
10^8	$7{,}0\times10^{-5}$	$1{,}64\times10^{-4}$

Con solo 20 repeticiones el RMS medido también tiene ruido (por eso en 10^8 salió algo por debajo del teórico), pero el orden de magnitud y la tendencia se cumplen.

I.2. ¿Por qué es un problema embarazosamente paralelo?

Un problema es embarazosamente paralelo cuando se puede partir en tareas que no necesitan comunicarse entre sí mientras trabajan. Al revisar el ciclo del programa secuencial vi que cada iteración i genera su propio par (x, y), evalúa $x^2 + y^2 \leq 1$ y, como mucho, suma 1 a un contador. Ninguna iteración lee un resultado producido por otra: no hay dependencias de flujo (lectura después de escritura) entre muestras, y el orden en que se procesan no cambia el resultado final, porque solo se cuentan aciertos. La única interacción es la acumulación final del contador, que es una suma (operación asociativa y conmutativa) y se resuelve con una reducción al final. Por eso el trabajo se puede repartir entre hilos casi sin sincronización.

Hay un detalle que no es obvio: en el código secuencial existe una dependencia oculta, que es el estado del generador (estado). Cada llamada a rand_r(&estado) lo modifica, así que si los hilos compartieran esa variable habría una condición de carrera. Esa dependencia es artificial (no viene del problema, sino de cómo se generan los números) y se elimina dándole a cada hilo su propio generador. Con eso, la clasificación de los datos queda así:

Variable	Atributo	Justificación
N	Compartida	Solo se lee; todos los hilos necesitan el mismo valor.
dentro (total)	Reducción	Cada hilo cuenta en una copia privada y al final OpenMP las suma con reduction(+:dentro).
i	Privada	Índice del ciclo; cada hilo recorre su propio bloque de iteraciones.
x, y	Privadas	Son temporales de cada muestra; compartirlas mezclaría coordenadas de hilos distintos.
estado (semilla)	Privada	Cada hilo necesita su propio flujo de números, independiente y sin solapamiento.

I.3. Repartir las muestras entre p hilos da el mismo estimador

Para demostrarlo, sea $X_i \in \{0,1\}$ el indicador de acierto de la muestra i, con i = 1, …, N. Al repartir las iteraciones entre p hilos, el conjunto de índices {1, …, N} queda dividido en p subconjuntos disjuntos S_1, …, Sp cuya unión es todo el conjunto (eso es exactamente lo que hace parallel for: cada iteración la ejecuta un único hilo). Si N no es divisible por p los tamaños Nk difieren, pero siempre $N_1 + \ldots + Np = N$. El hilo k calcula su conteo parcial $Dk = \Sigma i \in Sk\ X_i$, y la reducción los suma:

$$D_1 + \ldots + Dp = \Sigma k\ \Sigma i \in Sk\ X_i = \Sigma i=1..N\ X_i = D \quad \Rightarrow \quad 4 \cdot D/N = \pi N$$

La segunda igualdad se cumple porque los Sk forman una partición: cada término aparece exactamente una vez. Además, como la suma es de enteros (long), es exacta y no importa el orden en que la reducción combine los parciales; con números en punto flotante esto no sería exactamente cierto por el redondeo, pero aquí no hay ese problema. Por lo tanto, si los hilos recorren las mismas muestras que la versión secuencial, el resultado es idéntico bit a bit, y la división final $4 \cdot D/N$ es la misma que en el programa de un hilo.

Precisión esperada: con el mismo N total la precisión estadística es la misma que con un solo hilo, ni mejor ni peor. Aunque cada hilo use un flujo de números distinto, las N muestras siguen siendo independientes e idénticamente distribuidas, así que $E[\pi N] = \pi$ y $Var(\pi N) = 16q(1-q)/N$ no dependen de p. Es decir, el error sigue siendo $\approx 1{,}64 \times 10^{-4}$ para $N = 10^8$ con 1, 2, 4 u 8 hilos. Lo que cambia con p es solo el tiempo de ejecución. Si quiero más precisión tengo que aumentar N; lo que el paralelismo me permite es pagar ese aumento de N en menos tiempo. Esto es válido solo si los generadores de los hilos son de verdad independientes: si dos hilos repitieran la misma secuencia, las muestras dejarían de ser independientes y el error real sería mayor que el calculado

Parte II. Implementación en OpenMP (35 %)

Entorno y archivos. Ubuntu sobre WSL, GCC 13 con OpenMP, Intel Core i5-1235U (12 CPU lógicas). Archivos: montecarlo_secuencial.c (referencia del enunciado + medición con omp_get_wtime()), montecarlo.c (versión paralela), errores.c (versiones incorrectas a propósito, para II.3 y II.4) y ejecutar_parte2.sh, que compila con -Wall -Wextra (sin advertencias) y guarda todas las salidas en resultados_parte2.txt. Semilla base usada: 20261018.

II.1. Versión secuencial con N = 108

Obtuve $\hat{\pi}$ = 3,141633 (78 540 833 aciertos), con error $|\hat{\pi} - \pi|$ = 4,07×10−5. La estimación es consistente con π. El orden de magnitud del error sale de lo que deduje en la Parte I: la desviación estándar del estimador es $\sigma \approx 1{,}642/\sqrt{N}$, y con N = 108 da σ ≈ 1,64×10−4. Por eso un error del orden de 10−4 es lo normal; el mío quedó dentro de una desviación estándar (en otra semilla podría salir 1×10−4 o 2×10−4 y sería igual de correcto). Medí el tiempo solo del ciclo, con omp_get_wtime() antes y después, en 5 ejecuciones: mediana T(1) = 0,635 s (valores entre 0,627 y 0,649 s). Este valor lo conservo para la Parte III.

II.2. Versión paralela

El reto principal no fue el parallel for, sino las semillas: el enunciado pide flujos independientes, sin solapamiento, espaciados al menos N/p y que el resultado no dependa de p. Revisando el código fuente de glibc encontré que rand_r es un generador lineal congruencial de 32 bits: cada llamada aplica tres veces $s \leftarrow 1103515245 \cdot s + 12345 \pmod{232}$. Entonces cada muestra (x, y) consume exactamente 6 pasos, y la muestra i de la versión secuencial empieza en el paso 6i. Como un LCG se puede «adelantar» k pasos componiendo la transformación consigo misma (exponenciación binaria, O(log k)), escribí la función salto_lcg(semilla, k). Cuando un hilo empieza su bloque en la muestra i, salta directamente al estado 6i y desde ahí usa rand_r normal:

```
#pragma omp parallel for schedule(static) \
        firstprivate(estado, siguiente) reduction(+:dentro)
for (long i = 0; i < N; i++) {
   if (i != siguiente)              /* inicio de bloque: saltar */
      estado = salto_lcg(SEMILLA_BASE, 6ul * i);
   siguiente = i + 1;
   double x = rand_r(&estado) / (double)RAND_MAX;   /* igual que el original */
   double y = rand_r(&estado) / (double)RAND_MAX;
   if (x * x + y * y <= 1.0) dentro++;
}
```

Con schedule(static) cada hilo recibe un bloque contiguo de N/p muestras, así que cada uno salta una sola vez y su estado inicial queda separado exactamente N/p muestras (6N/p pasos) del siguiente; con N = 1000 y p = 4 el programa muestra que los hilos empiezan en las muestras 0, 250, 500 y 750. Los bloques son disjuntos y 6N = 6×108 < 232 ≈ 4,3×109, así que el generador no da la vuelta y no hay solapamiento. El número de hilos se elige con ./montecarlo N p (omp_set_num_threads) o con OMP_NUM_THREADS. El resultado es idéntico bit a bit para cualquier p, y además igual al secuencial (los tiempos, de 0,643 s con p = 1 a 0,109 s con p = 8, los analizo en la Parte III):

Secuencial	p = 1	p = 2	p = 3 (variable)	p = 4	p = 8 / 12
78 540 833	78 540 833	78 540 833	78 540 833	78 540 833	78 540 833

Todas las ejecuciones dan $\hat{\pi}$ = 3,1416333200. Limitación: el salto depende de la implementación de rand_r de glibc (la de Linux/WSL); en otro sistema habría que cambiar las constantes del LCG.

II.3. ¿Por qué no usar rand() con el estado global?

Problema 1, condición de carrera sobre el estado: rand() guarda su estado en una variable global. Cada llamada hace leer-calcular-escribir, y si dos hilos lo hacen a la vez pueden leer el mismo estado (y producir el mismo número) o pisarse la escritura. El estándar no garantiza que rand() sea seguro con hilos. En glibc, rand() tiene un candado interno, así que no se corrompe, pero los hilos se turnan: el orden en que cada hilo recibe los números cambia en cada ejecución (resultado no reproducible) y el candado serializa el programa. Con rand_r sobre una sola variable compartida (modo 2) no hay candado y sí hay carrera real (además, el tiempo varió de 0,64 a 4,3 s porque los hilos se disputan la misma línea de caché): números repetidos o saltados, la secuencia deja de ser la del generador y las muestras dejan de ser independientes, por lo que el error reportado ($\sigma \approx 1{,}64\times10-4$) ya no es confiable.

Problema 2, misma semilla en todos los hilos: si cada hilo tiene su estado pero lo inicializa con la misma semilla, todos generan la misma secuencia. Con p hilos solo hay N/p muestras distintas, cada una repetida p veces. El programa cree que usó N muestras, pero el error real es el de N/p, es decir, $\sqrt{p}$ veces mayor que el reportado (con p = 4, el doble). Es el error más peligroso porque no se nota: el resultado es estable y razonable.

La evidencia de errores.c (N = 108, p = 4, 3 ejecuciones) confirma las dos cosas:

Versión	π̂ en 3 ejecuciones	Tiempo	Diagnóstico
1: rand() global (glibc)	3,141813 / 3,141748 / 3,141758	7,4 – 9,8 s	Cambia cada vez; ~14× más lento que el secuencial
2: rand_r con estado compartido	3,139516 / 3,142034 / 3,139346	0,64 – 4,3 s	Cambia cada vez; error hasta $2{,}3\times10^{-3}$ (14σ)
3: misma semilla por hilo	3,141893 (igual las 3)	0,23 – 0,25 s	Idéntico a 1 hilo con N/4

El modo 3 da exactamente 19 636 834 × 4 = 78 547 336 aciertos, el mismo π̂ que una ejecución de un hilo con $N/4 = 2{,}5\times10^{7}$ muestras. Eso prueba que las muestras de los 4 hilos son copias.

II.4. Atributos de compartición de datos

En OpenMP, las variables declaradas fuera de la región paralela son compartidas por defecto, y las declaradas dentro del cuerpo del ciclo son privadas. El índice del parallel for es siempre privado. En mi código:

Variable	Por defecto → en mi código	Razón
N	Compartida → compartida	Solo se lee.
SEMILLA_BASE	Constante (macro)	No es variable; no hay nada que compartir.
i	Privada (índice del for)	Cada hilo recorre su bloque.
x, y	Privadas (declaradas dentro)	Temporales de cada muestra.
estado, siguiente	Compartidas → firstprivate	Copia propia por hilo, inicializada con el valor previo.
dentro	Compartida → reduction(+:dentro)	Contador privado por hilo, sumado al final.
p	Compartida	Solo la escribe la iteración i = 0 (un único hilo).

La cláusula reduction(+:dentro) crea una copia privada de dentro en cada hilo, inicializada en 0 (el neutro de la suma). Cada hilo cuenta sus aciertos sin interferir y al final OpenMP suma las copias en la variable original. Si quito la cláusula, dentro++ sobre la variable compartida es una condición de carrera (data race): la operación no es atómica, es leer, sumar 1 y escribir. Dos hilos pueden leer el mismo valor y ambos escribir valor+1, así que se pierde un

incremento (lost update). El efecto es que se cuentan menos aciertos de los reales, π sale subestimado y además cambia en cada ejecución. En mi prueba (modo 4, p = 4) obtuve $\hat{\pi}$ = 0,812 / 0,822 / 0,952: solo se registraron entre 20,3 y 23,8 millones de los ≈ 78,5 millones de aciertos, es decir, se perdió cerca del 70–74 % de los incrementos. Además, fue más lento, porque los hilos se disputan la misma línea de caché.

II.5. (Opcional) schedule(static) frente a schedule(dynamic)

Planificación (p = 4)	Tiempo en 3 ejecuciones	$\hat{\pi}$
static	0,233 / 0,248 / 0,250 s	3,1416333200
dynamic (chunk = 1)	4,625 / 4,598 / 4,484 s	3,1416333200
dynamic, 65536	0,239 / 0,240 / 0,245 s	3,1416333200

El resultado es el mismo en los tres casos gracias al salto del generador, que no depende de qué hilo tome cada iteración. En el tiempo sí hay diferencia: con dynamic de chunk 1 cada iteración es un bloque nuevo, el hilo pide trabajo a una cola compartida y tiene que hacer el salto O(log i) en cada muestra, así que fue unas 19 veces más lento (≈ 4,6 s frente a ≈ 0,24 s). Con chunk 65536 ese costo se reparte y el tiempo es prácticamente igual al de static (≈ 0,24 s). Mi conclusión es que en este problema la planificación no aporta nada: todas las iteraciones cuestan lo mismo, así que no hay desbalance que dynamic pueda corregir, y static es la opción natural porque no tiene costo de coordinación.

Parte III. Evaluación de desempeño: speedup y ley de Amdahl (30 %)

Protocolo experimental. Equipo: Intel Core i5-1235U, Ubuntu sobre WSL, 12 CPU lógicas (salida de lscpu en lscpu.txt). Usé N = 10^8 fijo, la versión paralela de la Parte II con schedule(static) y p ∈ {1, 2, 4, 8, 12}; agregué p = 12 porque es el número de CPU lógicas de mi equipo. El script ejecutar_parte3.sh hace una ejecución de calentamiento que no registro y luego 7 rondas; en cada ronda mide todos los p una vez, en lugar de hacer las 7 ejecuciones de un p seguidas, para que si el portátil se calienta y baja la frecuencia eso afecte a todos los p por igual. Además verifiqué que todas las ejecuciones dieran los mismos 78 540 833 aciertos.

¿Por qué la mediana y no el promedio? Los tiempos no se distribuyen de forma simétrica: el programa nunca puede correr más rápido que su costo real, pero sí más lento si Windows, la máquina virtual de WSL u otro programa le quitan el procesador un momento. Esas interrupciones producen valores atípicos solo hacia arriba, y el promedio se va detrás de ellos, mientras que la mediana los ignora y representa la ejecución típica. Con pocas repeticiones (5 a 7) un solo valor atípico cambia mucho el promedio y casi nada la mediana. En mis datos se ve claramente: en la ronda 3 algo interfirió y p = 4, 8 y 12 tardaron 0,205, 0,160 y 0,179 s, frente a medianas de 0,182, 0,110 y 0,116 s. Con p = 8, ese único valor sube el promedio a 0,117 s, mientras que la mediana se queda en 0,110 s.

p (hilos)	Mediana de T(p) [s]	Mín – máx [s]	S(p) = T(1)/T(p)	E(p) = S(p)/p	Karp–Flatt e
1	0,6227	0,609 – 0,652	1,000	1,000	—
2	0,3237	0,321 – 0,334	1,924	0,962	0,0397
4	0,1821	0,178 – 0,205	3,420	0,855	0,0566
8	0,1100	0,107 – 0,160	5,661	0,708	0,0590
12	0,1157	0,089 – 0,178	5,382	0,449	0,1118

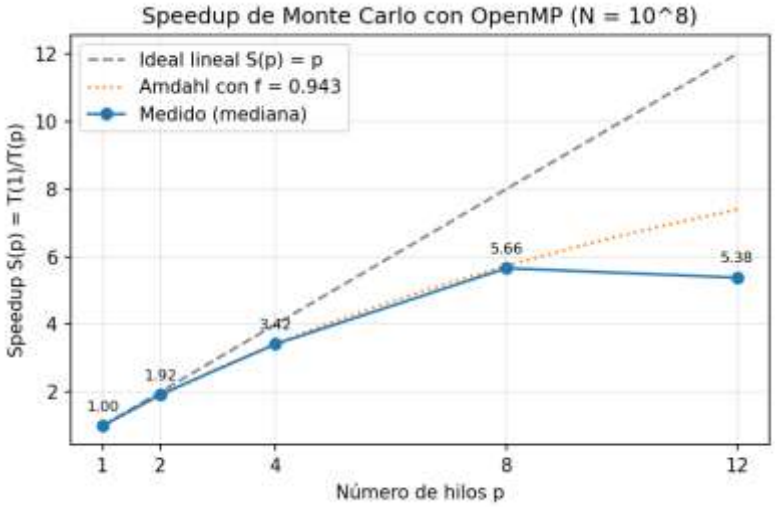


Figura 1. Speedup medido frente al ideal lineal y a la predicción de Amdahl con el f estimado en III.2.

III.1. Ley de Amdahl en tres casos

(a) f = 0,98, p = 8: S = 1 / (0,02 + 0,98/8) = 1 / (0,02 + 0,1225) = 7,02. Aunque el 98 % es paralelo, se pierde casi un hilo completo: eficiencia de 0,88. (b) f = 0,90, p = 8: S = 1 / (0,10 + 0,1125) = 4,71, eficiencia de 0,59; pasar del 2 % al 10 % secuencial hace que más de 3 de los 8 hilos se «desperdicien». (c) f = 0,98, p → ∞: el término f/p tiende a 0 y S → 1/(1 − f) = 1/0,02 = 50. Con infinitos núcleos el programa nunca será más de 50 veces más rápido, porque la parte secuencial sigue tardando lo mismo. Lo que me enseña el caso (c) es que la fracción secuencial, aunque sea pequeña, pone el techo; si quiero escalar a muchos núcleos, reducir 1 − f vale más que agregar núcleos.

III.2. Estimación de la fracción secuencial con p = 4

Despejo f de la ley de Amdahl: 1/S = (1 − f) + f/p ⇒ 1/S − 1 = f·(1/p − 1) ⇒ f = (1 − 1/S) / (1 − 1/p). Con mi medición S(4) = 3,420: 1/S = 0,2924, entonces f = (1 − 0,2924) / (1 − 0,25) = 0,7076 / 0,75 = 0,9434. Es decir, el programa se comporta como si el 94,3 % fuera paralelo y el 5,7 % secuencial. La métrica de Karp–Flatt e = (1/S − 1/p)/(1 − 1/p) mide la fracción secuencial «experimental»: con p = 4 da exactamente 1 − f = 0,0566, porque es el mismo despeje. Lo útil es calcularla para varios p (última columna de la tabla): si e se mantuviera constante, la pérdida vendría de código realmente secuencial; en mis datos e crece de 0,040 (p = 2) a 0,112 (p = 12), lo que indica que la pérdida aumenta con el número de hilos y se debe a sobrecostos del paralelismo y del hardware, no a una parte secuencial fija. De hecho, en mi código lo secuencial es mínimo (leer argumentos, la división final y el printf).

III.3. Medido frente a ideal y frente a Amdahl

Hasta p = 4 el speedup medido está cerca del ideal (S(2) = 1,92, S(4) = 3,42). Desde ahí se separa: con p = 8, ideal 8, Amdahl 5,73, medido 5,66; con p = 12, ideal 12, Amdahl 7,40, medido 5,38. El mejor resultado fue con p = 8. Analizo los factores uno por uno:

Creación de hilos y reducción: crear el equipo de hilos y combinar p enteros en la reducción cuesta microsegundos, frente a tiempos de ≈ 0,1 s o más. No explican la diferencia.

Memoria y caché: tampoco son el cuello de botella. Cada hilo solo maneja su estado del generador, x, y y su contador privado, que caben en registros; el programa no recorre arreglos, así que no depende del ancho de banda de memoria. Además, gracias a reduction no hay false sharing sobre el contador (en la Parte II, el modo 4 mostró lo que pasa cuando sí lo hay).

Hiperprocesamiento y núcleos híbridos (la causa principal): el i5-1235U tiene 2 núcleos de rendimiento (P) con hiperprocesamiento y 8 núcleos de eficiencia (E) sin él: 10 núcleos físicos y 12 hilos. WSL los muestra como «6 núcleos

× 2 hilos», que no es la topología real. Los núcleos E son más lentos que los P, y dos hilos sobre un mismo núcleo P comparten sus unidades de ejecución, así que no rinden el doble. Con schedule(static) cada hilo recibe N/p muestras, sin importar si cae en un núcleo rápido o lento, y el tiempo total lo marca el hilo más lento.

Frecuencia variable: con un solo hilo activo el procesador sube a su frecuencia turbo máxima (hasta 4,4 GHz en un núcleo P), pero con todos los núcleos ocupados baja la frecuencia por límites de potencia y temperatura de un portátil de 15 W. Eso hace que T(1) sea «favorable» y que el speedup con muchos hilos se vea menor.

Para comprobar lo del desbalance hice una prueba extra (5 ejecuciones): con p = 12, schedule(static) dio S = 5,28 y schedule(dynamic,65536) dio S = 6,53; con p = 8, 5,24 frente a 5,54. La mejora con dynamic en p = 12 confirma que parte de la pérdida viene de que static le da el mismo trabajo a núcleos de distinta velocidad: los hilos rápidos terminan y esperan al más lento. También es llamativo que con p = 12 los tiempos fueron los más dispersos de toda la tabla, entre 0,089 y 0,179 s, casi agrupados en dos niveles (≈ 0,09 s y ≈ 0,12 s). Mi interpretación es que depende de cómo Windows reparte los 12 hilos de la máquina virtual entre núcleos P y E en cada ejecución: cuando el reparto es favorable, p = 12 es el más rápido de todos (S ≈ 7,0 con 0,089 s).

Por eso el modelo de Amdahl, que supone p procesadores idénticos y una fracción secuencial fija, no describe bien mi equipo con más de 4 hilos: la «f» que estimé no es código secuencial, sino una forma de resumir todos estos sobrecostos en un solo número.

III.4. ¿Y con 64 núcleos?

Con mi f = 0,9434, Amdahl predice S(64) = 1 / (0,0566 + 0,9434/64) = 14,0, y el máximo absoluto es 1/(1 − f) = 17,7. Es decir, 64 núcleos darían un speedup de alrededor de 14, muy por debajo de 64 (eficiencia ≈ 0,22). Hay que diferenciar bien las dos variables: aumentar p con N fijo solo reduce el tiempo; la estimación es exactamente la misma, bit a bit, como vi en la Parte II, y el error sigue siendo ≈ $1{,}64\times10^{-4}$. Aumentar N es lo único que mejora la precisión, como $1/\sqrt{N}$, pero aumenta el tiempo proporcionalmente. La forma inteligente de usar los 64 núcleos es combinar ambas: mantener el tiempo y aumentar N. Por ejemplo, si con 64 núcleos proceso 14 veces más muestras en el mismo tiempo, el error baja $\sqrt{14} \approx 3{,}7$ veces.

Parte IV. Informe sobre su proceso de solución (20 %)

Trabajé todo en Ubuntu sobre WSL, en un portátil con Intel Core i5-1235U, y llevé el proyecto en un repositorio Git con un commit por parte. Me apoyé en Claude (Anthropic) como herramienta de IA durante todo el trabajo; el detalle de cada consulta está en la bitácora del anexo.

1. Decisiones de diseño

- **Estructura del programa.** Conservé el ciclo del programa de referencia sin cambios en su cuerpo (generar x e y con rand_r, probar $x^2 + y^2 \le 1$, contar) y solo agregué la directiva y el manejo del estado. Así la versión paralela se puede comparar directamente con la secuencial.
- **Cláusulas de OpenMP.** parallel for con reduction(+:dentro) para el contador; firstprivate(estado, siguiente) para que cada hilo tenga su propio generador; schedule(static) porque todas las iteraciones cuestan lo mismo. El número de hilos se elige como argumento (omp_set_num_threads) o con OMP_NUM_THREADS, y la semilla con la macro SEMILLA_BASE para poder cambiarla al compilar.
- **Semillas.** Fue la decisión más importante. La opción simple, semilla_k = base + k·N/p, no cumple dos requisitos: con rand_r, que dos semillas estén separadas por N/p como números no garantiza que sus secuencias no se solapen, y el resultado cambiaría con p porque cada p usaría muestras distintas. Como en glibc rand_r es un generador lineal congruencial de 32 bits (3 pasos por llamada, 6 por muestra), implementé salto_lcg(), que adelanta el generador k pasos en O(log k). Cada hilo salta al paso 6i de la misma secuencia de la versión secuencial, así que los flujos son disjuntos, quedan separados exactamente N/p muestras y el resultado no depende de p. El costo es que esto depende de la implementación de glibc.

2. Problemas que aparecieron y cómo los diagnostiqué

- **Distinguir cada defecto.** En lugar de solo describir los errores típicos, programé versiones incorrectas a propósito (errores.c) para ver su «firma» en los datos. Con rand_r compartido, π cambiaba en cada ejecución y el error llegó

a $2{,}3\times10^{-3}$, unas 14 desviaciones estándar. Con la misma semilla en todos los hilos, el resultado era estable y razonable (3,141893), lo que lo hace peligroso; la evidencia fue que daba exactamente 78 547 336 = 4 × 19 636 834 aciertos, igual que un hilo con N/4 muestras. Sin reduction, π cayó a 0,81: se perdió cerca del 74 % de los incrementos.

- **dynamic 19 veces más lento.** Con schedule(dynamic) y chunk 1 el tiempo pasó de ≈ 0,24 s a ≈ 4,6 s. El diagnóstico fue que con chunk 1 cada iteración es un bloque nuevo y el hilo ejecuta salto_lcg() en cada muestra, además de pedir trabajo a la cola. Con chunk 65536 el tiempo volvió a ≈ 0,24 s, lo que confirmó la causa.
- **p = 12 más lento que p = 8.** En la corrida única de la Parte II, p = 12 tardó 0,121 s y p = 8 0,109 s. lscpu decía «6 núcleos × 2 hilos», pero averigüé que el i5-1235U es híbrido: 2 núcleos P con hiperprocesamiento y 8 núcleos E. Mi hipótesis fue que static le da el mismo trabajo a núcleos rápidos y lentos, así que agregué una prueba en la Parte III: con p = 12, static dio S = 5,28 y dynamic,65536 dio S = 6,53. Eso confirmó la hipótesis.
- **Problemas del entorno.** git commit falló con «not a git repository» porque nunca había hecho git init; lo inicialicé y separé los commits por parte. También decidí compilar y medir dentro del sistema de archivos de Linux (~/montecarlo) y no en D: (/mnt/d), que en WSL es más lento, y le agregué al script un paso que corrige los saltos de línea de Windows (sed 's/\r$//').

3. Cómo validé los resultados

Para la estimación: $\pi\hat{} = 3{,}1416333$ con error $4{,}07\times10^{-5}$, menor que la desviación esperada $\sigma \approx 1{,}642/\sqrt{N} = 1{,}64\times10^{-4}$. Para respaldar esa σ, en la Parte I medí el error cuadrático medio con 20 semillas y siguió la ley $1/\sqrt{N}$ (por ejemplo, $1{,}53\times10^{-3}$ medido frente a $1{,}64\times10^{-3}$ teórico con $N = 10^6$). Para la reproducibilidad: el número de aciertos es una huella exacta, así que comparé los 78 540 833 aciertos entre la versión secuencial y todas las ejecuciones paralelas (p = 1, 2, 3, 4, 8 y 12, más las 35 mediciones de la Parte III y las tres planificaciones), y siempre coincidieron. Con la opción DETALLE verifiqué que, con N = 1000 y p = 4, los hilos empiezan en las muestras 0, 250, 500 y 750. Por último, compilé con -Wall -Wextra sin advertencias.

4. Conexión con mis datos

Mi mediana con p = 4 fue 0,1821 s: S = 3,42 y E = 0,855. Ese 85 % es lo que define mi f = 0,943, y aun así no hay código secuencial que explique un 5,7 %; Karp–Flatt sube de 0,040 (p = 2) a 0,112 (p = 12), así que la pérdida crece con los hilos. Con p = 4 el sistema ya tiene que usar el hiperprocesamiento o los núcleos E, porque solo hay 2 núcleos P, y con 4 hilos activos el turbo baja respecto a T(1), que se benefició de la frecuencia máxima. p = 8 fue el mejor caso con static (S = 5,66), y p = 12 fue el más irregular: entre 0,089 y 0,179 s, con S = 7,0 en su mejor ejecución. La mediana también me protegió de la ronda 3, en la que algo del sistema interfirió (p = 8 tardó 0,160 s frente a una mediana de 0,110 s).

5. Reflexión final

Lo que más aprendí es que en Monte Carlo paralelizar el ciclo es fácil y lo difícil es el generador aleatorio: un error ahí no rompe el programa, sino que produce un número razonable y equivocado, como en el caso de la semilla repetida. También aprendí que medir bien exige un protocolo (calentamiento, rondas intercaladas, mediana) y que la «f» de Amdahl es efectiva y resume todo lo que el hardware hace. Aprendí además a separar las dos variables: p reduce el tiempo y N mejora la precisión.

En un próximo trabajo paralelo cambiaría tres cosas: usaría desde el principio un generador pensado para paralelismo (por ejemplo, uno basado en contador, como Philox, o uno con salto documentado) en lugar de depender de los detalles internos de glibc; usaría schedule(dynamic) con bloques grandes o fijaría los hilos con OMP_PROC_BIND/OMP_PLACES para estudiar por separado los núcleos P y E; y crearía el repositorio Git desde el primer minuto.

Parte V. Dependencias entre iteraciones

En esta parte se estudia qué ocurre cuando una iteración de un bucle necesita datos que produce otra iteración. Primero se paraleliza, a propósito, la suma prefijo, que tiene una dependencia entre iteraciones; después se clasifican seis bucles según si sus iteraciones son independientes y se comprueban experimentalmente tres de ellos. Como extra, se muestra la forma correcta de paralelizar la suma prefijo con la directiva scan. El resultado principal: #pragma omp parallel for no revisa si el bucle se puede paralelizar; si hay dependencias, el programa compila sin avisos y entrega un resultado incorrecto.

Entorno de pruebas: Ubuntu sobre WSL, gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1), _OPENMP = 201511, procesador 12th Gen Intel(R) Core(TM) i5-1235U con 12 núcleos lógicos. Todo se ejecutó con el script ejecutar_parte5.sh, que guarda la salida completa en resultados_parte5.txt.

5.1 Un bucle que no se puede paralelizar

La **suma prefijo** acumula cada elemento sobre el anterior: después del bucle, a[i] debe contener la suma de a[0] hasta a[i]. Como todos los elementos empiezan en 1, el resultado correcto es a[i] = i + 1, y por lo tanto a[N − 1] = N = 1 000 000.

Programa prefijo.c:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
   const int N = 1000000;
   long *a = malloc(N * sizeof(long));
   for (int i = 0; i < N; i++) a[i] = 1;        // todos valen 1

   #pragma omp parallel for
   for (int i = 1; i < N; i++)
      a[i] = a[i - 1] + a[i];                   // usa el resultado anterior

   printf("a[N-1] = %ld (esperado %d)\n", a[N - 1], N);
   free(a);
   return 0;
}
```

Compilación y ejecución con 1, 2, 4, 8 y 12 hilos (agregué 12 porque es el número de núcleos lógicos de mi equipo):

```
gcc -Wall -fopenmp prefijo.c -o prefijo
for p in 1 2 4 8 12; do OMP_NUM_THREADS=$p ./prefijo; done
```

```
sapc1207@DESKTOP-GPVU5MM:~/taller-openmp/parte5$ gcc -Wall -fopenmp prefijo.c -o prefijo
sapc1207@DESKTOP-GPVU5MM:~/taller-openmp/parte5$ for p in 1 2 4 8 12; do OMP_NUM_THREADS=$p ./prefijo
 done
a[N-1] = 1000000 (esperado 1000000)
a[N-1] = 500000 (esperado 1000000)
a[N-1] = 250000 (esperado 1000000)
a[N-1] = 125000 (esperado 1000000)
a[N-1] = 83334 (esperado 1000000)
```

Figura 17. Compilación de prefijo.c con -Wall -fopenmp y ejecución con distinto número de hilos.

Hilos	a[N−1] obtenido	Esperado	¿Correcto?
1	1 000 000	1 000 000	**Sí**
2	500 000	1 000 000	**No**
4	250 000	1 000 000	**No**
8	125 000	1 000 000	**No**
12	83 334	1 000 000	**No**

Tabla 10. Resultado de la suma prefijo según el número de hilos.

Se ve un patrón muy claro: con p hilos el resultado es aproximadamente N / p (por ejemplo, 250 000 con 4 hilos y 83 334 con 12). Esto no es casualidad, y se explica en la Pregunta 2.

Pregunta 1. ¿Con cuántos hilos el resultado es correcto? ¿El compilador o el programa dieron algún aviso?

Respuesta: solo con 1 hilo. Ni el compilador ni el programa dieron ningún aviso.

La compilación con -Wall -fopenmp terminó sin mensajes y el programa se ejecutó normalmente: no se detuvo ni mostró errores, simplemente imprimió un valor equivocado. La única forma de notar el fallo es que el propio programa imprime el valor esperado al lado.

Esto pasa porque OpenMP **confía en el programador**. Escribir #pragma omp parallel for es una promesa: «las iteraciones de este bucle son independientes y se pueden ejecutar en cualquier orden». El compilador no comprueba esa promesa; solo reparte las iteraciones entre los hilos. Si la promesa es falsa, el error es **silencioso**, que es el tipo de error más peligroso. Además, que funcione con 1 hilo no demuestra nada: con un solo hilo las iteraciones se ejecutan en orden, igual que en el programa secuencial.

Pregunta 2. Explique por qué el resultado es incorrecto: ¿qué valor de a[i − 1] lee el primer hilo que trabaja en el segundo bloque?

Respuesta: lee el valor inicial 1, en vez del valor acumulado que debería tener. Ese hilo empieza a trabajar al mismo tiempo que el hilo 0, cuando el hilo 0 todavía no ha llegado al final de su bloque, así que la cadena de sumas se «reinicia» en cada bloque.

El bucle tiene una **dependencia de flujo entre iteraciones** (lectura después de escritura): la iteración i lee a[i − 1], que es justamente lo que escribe la iteración i − 1. Es una cadena: para calcular un elemento hay que esperar al anterior. Sin embargo, omp for reparte las 999 999 iteraciones (i = 1 hasta N − 1) en **bloques contiguos** y todos los hilos arrancan a la vez. Con 2 hilos:

Hilo	Iteraciones	Lee al empezar	Debería leer
0	i = 1 … 500 000	a[0] = 1	a[0] = 1 (correcto)
1	i = 500 001 … 999 999	a[500 000] = **1**	a[500 000] = **500 001**

Tabla 11. Qué lee cada hilo al empezar su bloque (2 hilos).

El hilo 1 lee a[500 000] en su primera iteración, pero ese elemento lo escribe el hilo 0 en su **última** iteración. En ese momento todavía vale 1, así que el hilo 1 calcula 2, 3, 4… como si el arreglo empezara de nuevo. El último elemento queda en 1 + 499 999 = 500 000. Lo mismo ocurre con más hilos: el valor final depende

solo del tamaño del último bloque.

Hilos	Último bloque	Lee a[ini − 1]	Debería ser	a[N−1] = 1 + iteraciones
2	i = 500 001 … 999 999	1	500 001	1 + 499 999 = **500 000**
4	i = 750 001 … 999 999	1	750 001	1 + 249 999 = **250 000**
8	i = 875 001 … 999 999	1	875 001	1 + 124 999 = **125 000**
12	i = 916 667 … 999 999	1	916 667	1 + 83 333 = **83 334**

Tabla 12. Predicción del resultado a partir del reparto por bloques.

Con 12 hilos, 999 999 / 12 = 83 333,25, así que GCC le da 83 334 iteraciones a los primeros 3 hilos y 83 333 a los demás; el último bloque tiene 83 333 iteraciones y por eso el resultado es 83 334. Las predicciones de la tabla coinciden con lo que medí (Tabla 10).

Hay que aclarar que esto es una **condición de carrera**: el valor que lee cada hilo depende de qué tan rápido avanzó el hilo anterior. En la práctica casi siempre lee 1, porque el hilo anterior tarda mucho en llegar al final de su bloque, pero con otra carga del sistema podría leer un valor intermedio. En ningún caso el resultado es confiable.

5.2 Clasificar bucles

Las iteraciones de un bucle son **independientes** cuando ninguna iteración escribe una posición de memoria que otra iteración lee o escribe. Si no, aparece una dependencia, que puede ser de **flujo** (una iteración lee lo que otra escribió), **anti-dependencia** (una iteración lee algo que otra va a sobrescribir) o de **salida** (dos iteraciones escriben la misma posición).

Bucle	¿Independiente?	Justificación
a[i] = b[i] + c[i];	**Sí**	Cada iteración escribe solo su propio a[i] y lee b y c, que el bucle no modifica.
a[i] = a[i+1] + 1;	**No**	**Anti-dependencia**: la iteración i lee a[i+1], que la iteración i+1 sobrescribe. En paralelo, en el borde de cada bloque se puede leer el valor ya modificado.
a[i] = a[i] * 2;	**Sí**	Cada iteración lee y escribe únicamente su propio elemento.
a[2*i] = a[2*i+1];	**Sí**	Solo se escriben posiciones pares y solo se leen impares; ninguna posición escrita es leída por otra iteración.
b[i] = a[i-1] + a[i];	**Sí**	Se lee a[i−1], pero a no se modifica: el resultado va a b (el bucle debe empezar en i = 1).
s = s + a[i];	**No**	Todas las iteraciones leen y escriben la **misma variable** s. Es una dependencia de **reducción**: como la suma es asociativa, se corrige con reduction(+:s) (Parte 6).

Tabla 13. Clasificación de los seis bucles.

Comprobación experimental

Comprobé tres de los seis bucles: uno independiente, a[i] = a[i] * 2, como control, y los dos dependientes,

a[i] = a[i+1] + 1 y s = s + a[i]. Para cada bucle, el programa ejecuta la versión secuencial sobre el arreglo sec y la paralela sobre el arreglo par, con los mismos datos iniciales, y compara los resultados. Para la suma, todos los elementos valen 1, así que el resultado esperado es N = 1 000 000.

El código de clasificar.c es el mismo de la versión de referencia (se incluye completo en los archivos de la entrega). Compilación y ejecución, tres veces con 4 hilos y una con 12:

```
gcc -Wall -fopenmp clasificar.c -o clasificar
for r in 1 2 3; do OMP_NUM_THREADS=4 ./clasificar; echo "---"; done
OMP_NUM_THREADS=12 ./clasificar
```

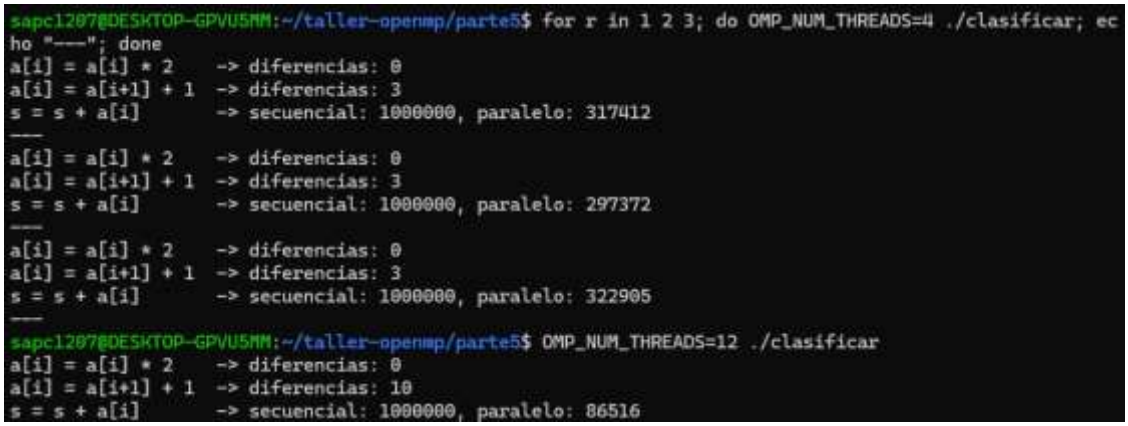


Figura 18. Ejecuciones de clasificar.c.

Bucle	Ejecución 1 (4 h)	Ejecución 2 (4 h)	Ejecución 3 (4 h)	12 hilos	¿Coincide con secuencial?
a[i] = a[i] * 2;	0 dif.	0 dif.	0 dif.	0 dif.	**Sí**
a[i] = a[i+1] + 1;	3 dif.	3 dif.	3 dif.	10 dif.	**No**
s = s + a[i];	s = 317 412	s = 297 372	s = 322 905	s = 86 516	**No**

Tabla 14. Resultado de cada ejecución.

Los resultados confirman la clasificación. a[i] = a[i] * 2 da **0 diferencias** en todas las ejecuciones: cada iteración solo toca su propio elemento, así que el orden no importa. Los otros dos bucles fallan siempre.

En a[i] = a[i+1] + 1 aparecieron exactamente 3 diferencias en las tres ejecuciones con 4 hilos, y no es casualidad: con 4 hilos el arreglo se divide en 4 bloques, que tienen **3 fronteras** entre sí. El problema solo ocurre en el último elemento de cada bloque: para calcularlo se lee el primer elemento del bloque siguiente, que pertenece a otro hilo y que ese hilo modifica en su primera iteración, mucho antes de que el hilo anterior llegue al final de su bloque. En el resto del bloque no hay problema, porque cada hilo recorre sus índices en orden y lee a[i+1] antes de sobrescribirlo.

Con 12 hilos hay 11 fronteras, pero obtuve 10 diferencias. Eso confirma que se trata de una condición de carrera y no de un error fijo: en una de las fronteras el hilo anterior alcanzó a leer a[i+1] antes de que el hilo siguiente lo modificara, probablemente porque ese hilo siguiente arrancó tarde (mi procesador tiene 12 hilos lógicos y el sistema no siempre los pone a correr todos al mismo instante). Por eso el número de

diferencias es «como máximo p − 1». La anti-dependencia se elimina escribiendo en un arreglo auxiliar (b[i] = a[i+1] + 1), con lo que el bucle se vuelve independiente.

En s = s + a[i] el error es enorme: con 4 hilos la suma paralela quedó entre 297 372 y 322 905, menos de la tercera parte de 1 000 000, y **cambia en cada ejecución**. Con 12 hilos fue peor: 86 516, menos del 9 %. Todas las iteraciones compiten por la misma variable s: cada hilo lee s, le suma 1 y lo guarda, y si otro hilo hizo lo mismo en medio, una de las dos sumas se pierde. Cuantos más hilos compiten, más sumas se pierden. Por eso el resultado siempre queda por debajo de N y nunca por encima. Este es el tema de la Parte 6, donde se corrige con reduction(+:s).

5.3 (Extra) La suma prefijo sí se puede paralelizar: directiva scan

La suma prefijo no se puede paralelizar «tal cual», pero sí con otro algoritmo: cada hilo suma su propio bloque, luego se calcula cuánto debe sumarle a cada bloque lo acumulado por los anteriores, y en una segunda pasada cada hilo corrige su bloque. OpenMP 5.0 lo automatiza con la directiva scan. Aunque la macro _OPENMP de mi compilador dice 201511 (OpenMP 4.5), GCC acepta scan desde la versión 10, porque implementa partes de la 5.0 sin haberla completado toda; por eso lo probé:

```
long acum = 0;
#pragma omp parallel for reduction(inscan, +:acum)
for (int i = 0; i < N; i++) {
  acum += a[i];              // fase 1: cada hilo suma su bloque
  #pragma omp scan inclusive(acum)
  a[i] = acum;               // fase 2: se escribe el prefijo corregido
}
```

Compilación y ejecución con 1, 2, 4, 8 y 12 hilos (también verifiqué la versión del compilador y el valor de _OPENMP):

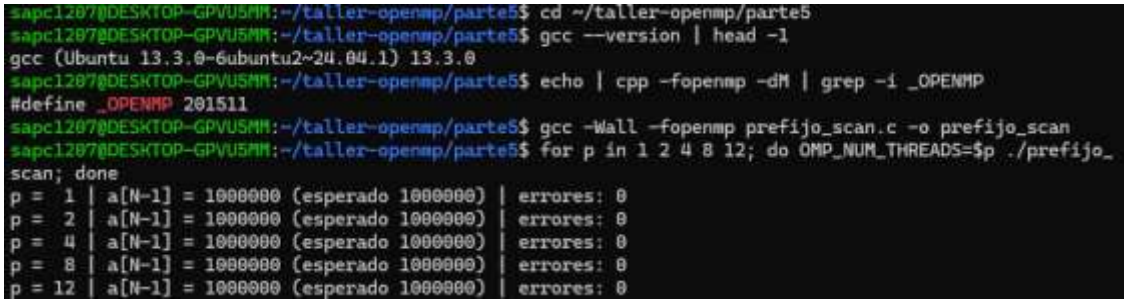


Figura 19. Versión de gcc, valor de _OPENMP y ejecución de prefijo_scan.c con distinto número de hilos.

Hilos	a[N−1]	Errores (de 10^6)	¿Correcto?
1	1 000 000	0	**Sí**
2	1 000 000	0	**Sí**
4	1 000 000	0	**Sí**
8	1 000 000	0	**Sí**
12	1 000 000	0	**Sí**

Tabla 15. Suma prefijo con scan: el programa revisa los 10^6 elementos, no solo el último.

Con scan el resultado es correcto con cualquier número de hilos, y el programa verifica todos los

elementos, no solo el último. Esto confirma que el problema de 5.1 no era la suma prefijo en sí, sino paralelizarla con un algoritmo que supone iteraciones independientes.

Parte VI. Condiciones de carrera y reduction

En esta parte se provoca a propósito una condición de carrera al sumar los elementos de un arreglo en paralelo, se corrige con la cláusula reduction y se usan varias reducciones a la vez. El resultado principal: sin reduction la suma paralela pierde incrementos y da un valor menor que N, distinto en cada ejecución; con reduction(+:suma) siempre da N. Las reducciones de punto flotante son correctas, pero los últimos dígitos dependen del número de hilos e incluso pueden cambiar entre ejecuciones con el mismo número de hilos.

Entorno de pruebas: Ubuntu sobre WSL, gcc 13.3.0 con la opción -fopenmp. Todo se ejecutó con el script ejecutar_parte6.sh, que guarda la salida completa en resultados_parte6.txt, y las figuras son capturas de las ejecuciones en la terminal. Equipo:

Dato	Valor en mi equipo
Modelo de procesador	12th Gen Intel(R) Core(TM) i5-1235U (15 W)
Núcleos físicos	10: 2 de rendimiento (P) con hiperprocesamiento + 8 de eficiencia (E) sin él
Núcleos lógicos (nproc)	12
Lo que reporta lscpu en WSL	6 núcleos × 2 hilos por núcleo (WSL no muestra la arquitectura híbrida)

Tabla 16. Datos del equipo usado en la Parte 6.

6.1 Provocar una condición de carrera

Todos los elementos del arreglo valen 1, así que la suma debería ser exactamente N = 10 000 000. Programa carrera.c:

```
/* carrera.c -- Parte 6.1: condicion de carrera A PROPOSITO.
   Todos los hilos suman sobre la misma variable sin proteccion. */
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(void) {
    const long N = 10000000;
    double *a = malloc(N * sizeof(double));
    for (long i = 0; i < N; i++) a[i] = 1.0;

    double suma = 0.0;
    #pragma omp parallel for
    for (long i = 0; i < N; i++)
        suma += a[i];              // varios hilos escriben la misma variable

    printf("suma = %.0f (esperado %ld)\n", suma, N);
    free(a);
    return 0;
}
```

Compilación y ejecución:

```
gcc -Wall -fopenmp carrera.c -o carrera
OMP_NUM_THREADS=1 ./carrera
for r in 1 2 3 4 5; do OMP_NUM_THREADS=4 ./carrera; done
```

```
sapc1207@DESKTOP-GPVU5MM:~/taller-openmp/parte6$ gcc -Wall -fopenmp carrera.c -o carrera
sapc1207@DESKTOP-GPVU5MM:~/taller-openmp/parte6$ OMP_NUM_THREADS=1 ./carrera
suma = 10000000 (esperado 10000000)
sapc1207@DESKTOP-GPVU5MM:~/taller-openmp/parte6$ for r in 1 2 3 4 5; do OMP_NUM_THREADS=4 ./carrera; d
one
suma = 3150788 (esperado 10000000)
suma = 2543893 (esperado 10000000)
suma = 2984027 (esperado 10000000)
suma = 3039325 (esperado 10000000)
suma = 2989438 (esperado 10000000)
```

Figura 20. Compilación de carrera.c y ejecución con 1 hilo y cinco veces con 4 hilos.

Con 1 hilo el resultado es exacto: suma = 10 000 000. Con más hilos, cada ejecución da un valor distinto y siempre menor. Además de las cinco ejecuciones de la Figura 20, el script repitió la prueba con 4 y con 12 hilos:

Ejecución	1	2	3	4	5
4 hilos (Figura 20)	3 150 788	2 543 893	2 984 027	3 039 325	2 989 438
4 hilos (script)	2 217 850	2 429 726	2 482 521	2 663 081	2 851 864
12 hilos (script)	1 615 314	1 069 704	1 007 270	1 358 593	1 633 210

Tabla 17. Resultado de la suma sin reduction (esperado: 10 000 000).

Con 4 hilos se pierde entre el 68 % y el 78 % de los incrementos, y con 12 hilos entre el 84 % y el 90 %. Cuantos más hilos actualizan la misma variable al mismo tiempo, más escrituras se pisan entre sí.

1. ¿El resultado con 4 hilos es mayor o menor que N? ¿Por qué nunca es mayor?

Respuesta: siempre es menor, porque una carrera puede borrar incrementos, pero no puede contar uno dos veces.

La instrucción suma += a[i] no es una sola operación: el hilo lee suma de memoria, le suma 1 en un registro y escribe el resultado. Si otro hilo escribe entre esa lectura y esa escritura, la escritura del primero pisa la del segundo y ese incremento se pierde. Nunca es mayor porque cada escritura vale «lo que el hilo leyó + 1», y lo que leyó nunca supera los incrementos que realmente se hicieron. El valor cambia en cada ejecución porque depende de cómo el planificador intercale los hilos, como se vio en la Parte 1.2.

2. Construya una tabla de intercalado en la que dos hilos ejecutan tres incrementos en total y se pierden dos de ellos.

El hilo 0 hace un incremento y el hilo 1 hace dos. suma empieza en 0, así que el resultado correcto sería 3:

Paso	Hilo 0	Hilo 1	suma en memoria
1	lee suma → registro = 0		0
2		lee suma → registro = 0	0
3		registro = 0 + 1, escribe 1	1
4		lee suma → registro = 1	1

5		registro = 1 + 1, escribe 2	2
6	registro = 0 + 1, escribe 1		**1**

Tabla 18. Tabla de intercalado: se pierden dos de tres incrementos.

Se ejecutaron 3 incrementos, pero suma termina en 1: el hilo 0 leyó un valor viejo en el paso 1 y en el paso 6 sobrescribe los dos incrementos del hilo 1.

6.2 Corregir con reduction

Copié el programa en carrera_reduction.c, cambiando solo la directiva:

```
#pragma omp parallel for reduction(+:suma)
```

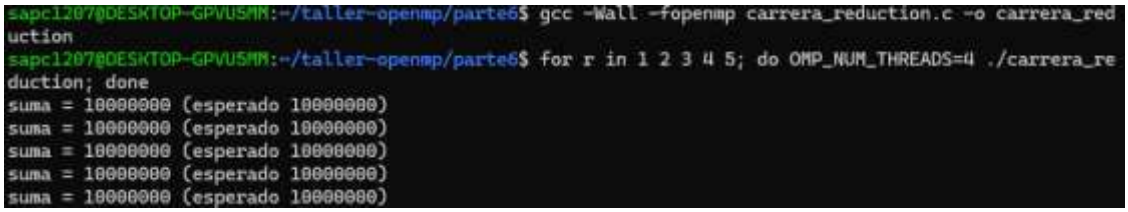


Figura 21. Ejecución de carrera_reduction.c cinco veces con 4 hilos.

1. Ejecute de nuevo cinco veces con 4 hilos. ¿El resultado es ahora siempre correcto?

Respuesta: sí. Las cinco ejecuciones con 4 hilos imprimen suma = 10000000 (esperado 10000000) (Figura 21). El script también lo ejecutó cinco veces con 12 hilos y el resultado fue siempre 10 000 000: la corrección no depende del número de hilos.

2. Describa con sus palabras los tres pasos que realiza OpenMP para implementar la reducción.

1. Copia privada. Al entrar al bucle, cada hilo crea su propia copia de suma y la inicializa con el elemento neutro del operador (0 para la suma).

2. Acumulación local. Cada hilo suma sus iteraciones en su copia privada. Como ningún otro hilo la toca, no hay carrera.

3. Combinación. Al final del bucle, OpenMP combina las copias parciales con el operador y suma el total a la variable original de forma sincronizada (un hilo a la vez), antes de la barrera implícita.

6.3 Varias reducciones a la vez

Se calcula en un solo bucle paralelo el máximo, el mínimo y la cantidad de elementos positivos del arreglo a[i] = sin(0.001 · i) · (i % 1000), con N = 10^7. El cuerpo del bucle es igual al de la versión secuencial; la cláusula reduction se encarga de que cada hilo tenga sus propias copias de mx, mn y positivos. Programa reducciones.c:

```
/* reducciones.c -- Parte 6.3: maximo, minimo y conteo en un solo bucle,
   con tres reducciones, comparado con la version secuencial. */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

int main(void) {
```

```
    const long N = 10000000;
    double *a = malloc(N * sizeof(double));
    for (long i = 0; i < N; i++)
        a[i] = sin(0.001 * i) * (i % 1000);

    /* Version secuencial (referencia) */
    double mx_s = -1e300, mn_s = 1e300;
    long pos_s = 0;
    for (long i = 0; i < N; i++) {
        if (a[i] > mx_s) mx_s = a[i];
        if (a[i] < mn_s) mn_s = a[i];
        if (a[i] > 0) pos_s++;
    }

    /* Version paralela con tres reducciones */
    double mx = -1e300, mn = 1e300;
    long positivos = 0;
    #pragma omp parallel for reduction(max:mx) reduction(min:mn) reduction(+:positivos)
    for (long i = 0; i < N; i++) {
        if (a[i] > mx) mx = a[i];
        if (a[i] < mn) mn = a[i];
        if (a[i] > 0) positivos++;
    }

    printf("Hilos: %d\n", omp_get_max_threads());
    printf("Maximo:    %.15f %.15f\n", mx_s, mx);
    printf("Minimo:    %.15f %.15f\n", mn_s, mn);
    printf("Positivos: %ld %ld\n", pos_s, positivos);
    free(a);
    return 0;
}
```

Compilación y ejecución (la captura de la Figura 22 incluye también la Parte 6.4):

```
gcc -Wall -fopenmp reducciones.c -o reducciones -lm
OMP_NUM_THREADS=4 ./reducciones
```

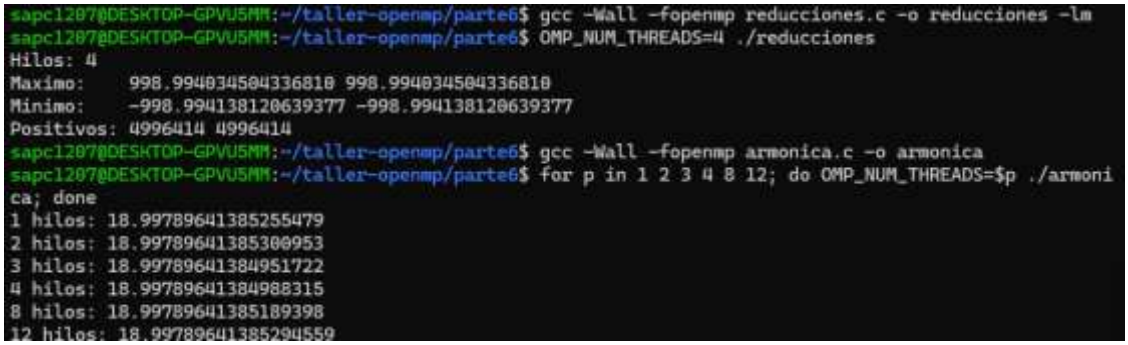


Figura 22. Ejecución de reducciones.c con 4 hilos y de armonica.c con 1, 2, 3, 4, 8 y 12 hilos.

	Secuencial	Paralelo (4 y 12 hilos)	¿Coincide?
Máximo	998.994034504336810	998.994034504336810	Sí
Mínimo	−998.994138120639377	−998.994138120639377	Sí
Positivos	4 996 414	4 996 414	Sí

Tabla 19. Resultado de las tres reducciones.

Los valores coinciden exactamente, con 4 y también con 12 hilos, porque máximo, mínimo y conteo son operaciones exactas: no hay redondeo, así que el orden en que se combinan los resultados parciales no importa. Los valores iniciales −1e300 y 1e300 cumplen el papel del elemento neutro, porque cualquier dato los reemplaza. La opción -lm es necesaria para enlazar la función sin: el script compiló a propósito sin ella y el enlazador falló con undefined reference to 'sin'.

6.4 Reducción y punto flotante

Programa armonica.c, que calcula la suma armónica $H(10^8)$ con una reducción:

```
/* armonica.c -- Parte 6.4: suma armonica H(10^8) con reduccion en punto flotante. */
#include <stdio.h>
#include <omp.h>

int main(void) {
   double h = 0.0;
   #pragma omp parallel for reduction(+:h)
   for (long i = 0; i < 100000000; i++)
      h += 1.0 / (i + 1);
   printf("%d hilos: %.17f\n", omp_get_max_threads(), h);
   return 0;
}
```

Compilación y ejecución (Figura 22):

```
gcc -Wall -fopenmp armonica.c -o armonica
for p in 1 2 3 4 8 12; do OMP_NUM_THREADS=$p ./armonica; done
```

Hilos	$H(10^8)$ obtenido	Diferencia con el valor real
1	18.99789641385255479	$1{,}3 \cdot 10^{-12}$
2	18.99789641385300953	$0{,}9 \cdot 10^{-12}$
3	18.99789641384951722	$4{,}4 \cdot 10^{-12}$
4	18.99789641384988315	$4{,}0 \cdot 10^{-12}$
8	18.99789641385189398	$2{,}0 \cdot 10^{-12}$
12	18.99789641385294559	$1{,}0 \cdot 10^{-12}$
4 (repetición 1)	18.99789641384988315	$4{,}0 \cdot 10^{-12}$
4 (repetición 2)	18.99789641384987959	$4{,}0 \cdot 10^{-12}$
4 (repetición 3)	18.99789641384988315	$4{,}0 \cdot 10^{-12}$

Tabla 20. Suma armónica según el número de hilos (valor real: $H(10^8) = 18.997896413853898\ldots$).

Reflexione: los resultados difieren en los últimos dígitos según el número de hilos. Si la reducción es correcta, ¿de dónde viene la diferencia? ¿Cuál de los resultados es el «correcto»?

Respuesta: la diferencia viene de que la suma en punto flotante no es asociativa; ninguno de los resultados es exacto y todos son igual de válidos.

La reducción es correcta: no se pierde ningún término. Pero (a + b) + c puede dar distinto que a + (b + c), porque cada suma se redondea a 53 bits de mantisa. Con p hilos, cada uno suma un bloque distinto y después se suman las p sumas parciales, así que el orden de las operaciones, y por tanto los redondeos, cambia con p.

El valor real de $H(10^8)$ es 18.997896413853898…, y todos los resultados difieren de él en menos de $5 \cdot 10^{-12}$. El de 1 hilo no es más exacto por ser secuencial: solo es otro orden de suma (de hecho, el de 2 hilos quedó más cerca).

Un detalle que encontré en mis datos: al repetir con 4 hilos, el resultado no siempre fue el mismo. Dos ejecuciones dieron …88315 y una dio …87959. El reparto de iteraciones sí es siempre igual, así que cada hilo calcula exactamente la misma suma parcial; lo que cambia es el **orden en que se combinan las sumas parciales al final**, que depende de qué hilo termina primero. Como la suma no es asociativa, ese orden también cambia el último dígito. Entonces el resultado depende sobre todo del número de hilos, pero también puede variar un poco entre ejecuciones; en cualquier caso, la diferencia (del orden de 10^{-15} entre ejecuciones y de 10^{-12} respecto al valor real) está muy por debajo de la precisión que se necesita.

Parte VII. Caso de estudio: π y medición de rendimiento

En esta parte se mide el rendimiento del cálculo de π. El resultado principal: en mi i5-1235U el speedup es casi ideal con 2 hilos (1,87), pero después crece despacio hasta 3,74 con 16 hilos; la versión paralela solo compensa desde $N = 10^6$.

El programa calcula π con la regla del punto medio sobre la integral de $4/(1 + x^2)$ entre 0 y 1, que vale exactamente π. Imprime cuatro columnas: hilos, tiempo de pared (s), tiempo de CPU (s) y error absoluto. Programa pi.c:

```
/* pi.c -- Parte 7: pi por la regla del punto medio sobre la integral de
   4/(1+x^2) entre 0 y 1. Imprime: hilos, tiempo de pared, tiempo de CPU, error. */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    const long N = (argc > 1) ? atol(argv[1]) : 200000000;
    const double dx = 1.0 / N;
    double suma = 0.0;

    clock_t c0 = clock();            // tiempo de CPU
    double t0 = omp_get_wtime();     // tiempo de pared

    #pragma omp parallel for reduction(+:suma)
    for (long i = 0; i < N; i++) {
        double x = (i + 0.5) * dx;
        suma += 4.0 / (1.0 + x * x);
    }
```

```
    double t1 = omp_get_wtime();
    clock_t c1 = clock();
    double pi = suma * dx;
    printf("%d %.6f %.6f %.3e\n", omp_get_max_threads(), t1 - t0,
        (double)(c1 - c0) / CLOCKS_PER_SEC, fabs(pi - M_PI));
    return 0;
}
```

Para las tres secciones usé el script medir_parte7.sh, que compila pi.c con -O2 -fopenmp -lm, hace una ejecución de calentamiento y luego ejecuta los mismos comandos de la guía. En 7.2 y 7.3 repite cada medición 5 veces y toma la mediana (ordena por la columna 2 y se queda con la tercera línea). Además mide con costo.c (Parte 2) el costo de fork-join en este mismo equipo. Las mediciones se hicieron con el portátil conectado al cargador y sin otros programas abiertos. Equipo: Intel Core i5-1235U (2 núcleos P con hiperprocesamiento + 8 núcleos E, 12 hilos lógicos), Ubuntu sobre WSL, gcc 13.3.0.

7.1 Medir bien el tiempo

```
gcc -O2 -fopenmp pi.c -o pi -lm
for p in 1 2 4 8; do OMP_NUM_THREADS=$p ./pi; done
```



Figura 23. Compilación de pi.c y ejecución con 1, 2, 4 y 8 hilos.

Hilos	Tiempo de pared (s)	Tiempo de CPU, clock() (s)	CPU / pared
1	0,208970	0,210220	1,0
2	0,124570	0,249384	2,0
4	0,088340	0,322592	3,7
8	0,068304	0,482920	7,1

Tabla 21. Tiempo de pared frente a tiempo de CPU (valores de la Figura 23).

Reflexione: ¿Qué le pasa al tiempo que mide clock() cuando aumenta el número de hilos? ¿Por qué no sirve para medir el beneficio de la paralelización?

Respuesta: el tiempo de clock() aumenta con los hilos, mientras que el tiempo de pared disminuye; clock() mide trabajo total, no el tiempo que se espera.

De 1 a 8 hilos el tiempo de pared bajó de 0,209 s a 0,068 s (3 veces menos), pero el de clock() subió de 0,210 s a 0,483 s (2,3 veces más). clock() mide el tiempo de CPU del proceso, es decir, la suma de lo que trabajó cada hilo. El trabajo total no disminuye al repartirlo; aquí incluso aumenta, porque los núcleos E son más lentos y gastan más tiempo de CPU en la misma cantidad de iteraciones, y porque con todos los núcleos ocupados la frecuencia baja.

Lo que la paralelización reduce es el tiempo que el usuario espera, y eso lo mide omp_get_wtime(). El cociente CPU / pared indica cuántos hilos estuvieron ocupados en promedio (0,483 / 0,068 ≈ 7,1 con 8 hilos), y sirve para comprobar que el paralelismo realmente ocurrió.

7.2 Speedup y eficiencia

```
for p in 1 2 4 8 12 16; do
  for r in 1 2 3 4 5; do OMP_NUM_THREADS=$p ./pi; done \
    | sort -n -k2 | sed -n 3p
done
```

Agregué p = 12 (los núcleos lógicos de mi equipo) a los valores de la guía.

Hilos (p)	Tiempo mediano Tp (s)	Speedup S = T1 / Tp	Eficiencia E = S / p
1	0,199243	1,00	1,00
2	0,106802	1,87	0,93
4	0,093611	2,13	0,53
8	0,067227	2,96	0,37
12	0,066725	2,99	0,25
16	0,053337	3,74	0,23

Tabla 22. Speedup y eficiencia (mediana de 5 ejecuciones, N = 2·10⁸).

1. Grafique el speedup en función de p junto con la recta ideal S = p.

La gráfica se generó con el script analizar_parte7.py (matplotlib) a partir de los datos de la tabla:

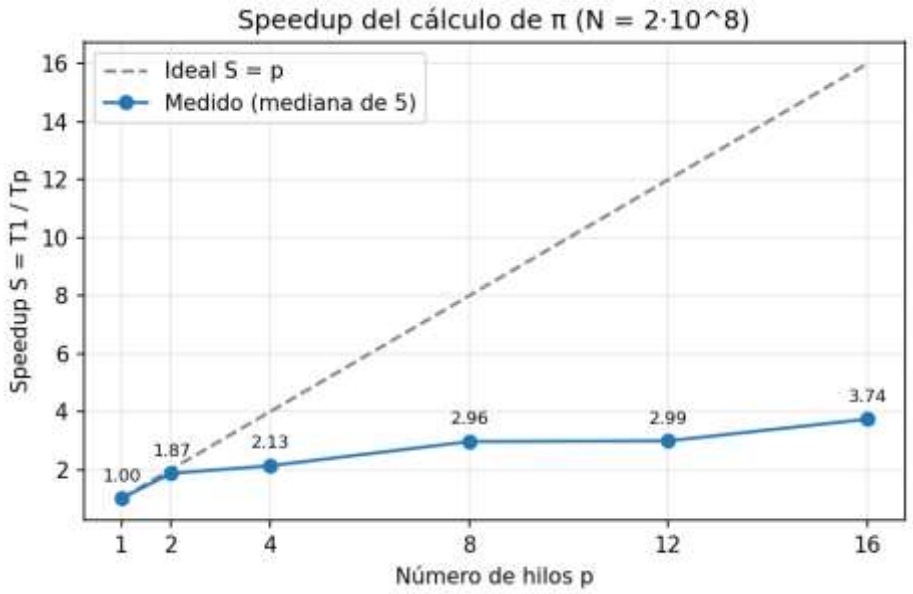


Figura 24. Speedup medido frente a la recta ideal S = p.

2. Compare con el número de núcleos físicos de la tabla de la Parte 0. ¿Qué ocurre con el speedup al pasar de los núcleos físicos a los lógicos?

Respuesta: en este procesador no hay un salto claro al pasar de físicos a lógicos, porque sus núcleos no son iguales: la curva se aplana desde 4 hilos, mucho antes de agotar los 10 núcleos físicos.

El i5-1235U tiene 2 núcleos P rápidos y 8 núcleos E más lentos (10 físicos, 12 lógicos). Con 2 hilos el speedup es 1,87, casi ideal: cada hilo corre en un núcleo P. Desde 4 hilos, parte del trabajo cae en núcleos E y la curva se aplana (2,13 con 4 y 2,96 con 8), aunque todavía quedan núcleos físicos libres. Al pasar de 8 a

12 hilos, es decir, al llenar los núcleos lógicos, el speedup prácticamente no cambia (2,96 → 2,99): los 4 hilos extra caen en el hiperprocesamiento de los núcleos P o en núcleos E y no aportan.

El motivo es que omp for reparte las iteraciones en partes iguales. Los hilos que corren en núcleos P terminan primero y esperan en la barrera al más lento, que está en un núcleo E. Con 16 hilos (más que los 12 lógicos) el speedup sí sube a 3,74: los bloques son más pequeños y el sistema operativo reparte varios hilos por núcleo, así que los núcleos rápidos terminan haciendo más trabajo y la carga se equilibra un poco mejor. Aun así, la eficiencia es de solo 0,23.

3. ¿Por qué la eficiencia tiende a disminuir al aumentar p?

Respuesta: porque el trabajo de cada hilo se reduce, pero el hilo más lento, el límite de consumo y la sobrecarga de OpenMP no se reducen en la misma proporción.

- **Núcleos desiguales.** Con reparto en partes iguales, el tiempo lo marca el hilo más lento. Con 4 hilos, si un núcleo E rinde una fracción r de un núcleo P, el hilo del núcleo E tarda T1 / (4r); con el S = 2,13 medido, r ≈ 0,53, es decir, un núcleo E rinde aproximadamente la mitad que uno P en este bucle.
- **Límite de consumo y frecuencia.** Es un procesador de 15 W. Con 1 hilo, el núcleo P sube a su turbo máximo; con muchos núcleos ocupados, la frecuencia de todos baja para no pasar del límite. T1 queda «favorecido» y el speedup aparente baja.
- **Recursos compartidos.** Los dos hilos de un núcleo P comparten sus unidades de cálculo (este bucle usa sobre todo la de división), y los núcleos E comparten caché en grupos de 4.
- **Sobrecarga y sobresuscripción.** Crear el equipo, la barrera y combinar la reducción cuestan más con más hilos (Tabla 24), mientras que el trabajo de cada hilo se achica. Con 16 hilos hay más hilos que núcleos lógicos y aparecen cambios de contexto.

7.3 ¿Cuándo vale la pena paralelizar?

```
for n in 1000 10000 100000 1000000 10000000 100000000; do
  echo "N = $n"
  OMP_NUM_THREADS=1 ./pi $n
  OMP_NUM_THREADS=8 ./pi $n
done
```

Resultados obtenidos (mediana de 5 ejecuciones; speedup = T con 1 hilo / T con 8 hilos):

N	T con 1 hilo (s)	T con 8 hilos (s)	Speedup	Error (1 hilo)
10^3	0,000005	0,000602	0,01	$8{,}333 \cdot 10^{-8}$
10^4	0,000015	0,000339	0,04	$8{,}333 \cdot 10^{-10}$
10^5	0,000106	0,000679	0,16	$8{,}368 \cdot 10^{-12}$
10^6	0,001032	0,000784	1,32	$2{,}887 \cdot 10^{-14}$
10^7	0,010735	0,004541	2,36	$6{,}217 \cdot 10^{-14}$
10^8	0,105031	0,035645	2,95	$6{,}333 \cdot 10^{-13}$

Tabla 23. Tiempo con 1 y 8 hilos según el tamaño del problema.

1. ¿A partir de qué tamaño empieza a compensar la versión paralela en su equipo?

Respuesta: desde N = 10^6, el primer tamaño con speedup mayor que 1 (1,32).

Con $N = 10^5$ o menos, la versión con 8 hilos es más lenta que la secuencial: con $N = 10^3$ tarda 0,60 ms contra 0,005 ms, unas 120 veces más. Con $N = 10^8$ el speedup llega a 2,95, prácticamente el mismo que en 7.2 con 8 hilos (2,96).

2. Relacione ese umbral con el costo del fork-join que midió en la Parte 2.

Respuesta: la versión paralela tiene un costo fijo de arranque; solo compensa cuando el trabajo de cada hilo es claramente mayor que ese costo.

Con 8 hilos el tiempo nunca baja de unos 0,34–0,68 ms ($N = 10^3$ a 10^5), aunque el trabajo real con 1 hilo sea de apenas 0,005–0,1 ms. Ese piso es el costo fijo de arrancar la región paralela. Para compararlo, medí el costo de fork-join en este mismo equipo con costo.c:

Hilos	1	2	4	8	12
µs por región	0,164	0,305	0,788	1,169	2,547

Tabla 24. Costo de abrir y cerrar una región paralela vacía en mi equipo (costo.c).

El piso de pi.c (cientos de µs) es unas 300 veces mayor que los 1,17 µs por región con 8 hilos de costo.c. La diferencia es que costo.c repite la misma región 100 000 veces y OpenMP reutiliza los hilos ya creados, mientras que cada ./pi es un proceso nuevo con una sola región: paga la creación de 8 hilos del sistema operativo, y en WSL además la de una máquina virtual. Paralelizar compensa cuando el trabajo de cada hilo (T1 / 8) supera claramente ese costo fijo. Con $N = 10^6$, T1 / 8 ≈ 0,13 ms es del mismo orden que el piso, y por eso el speedup apenas pasa de 1. Esta es la respuesta a la pregunta que quedó abierta en la Parte 2.

3. Observe la última columna de la salida: ¿cómo cambia el error de la aproximación de π con N?

Respuesta: primero baja 100 veces por cada factor 10 en N; desde $N \approx 10^6$ deja de bajar y aumenta un poco por el redondeo.

Entre $N = 10^3$ y 10^5 el error cae de $8{,}3 \cdot 10^{-8}$ a $8{,}3 \cdot 10^{-10}$ y luego a $8{,}4 \cdot 10^{-12}$: la regla del punto medio tiene un error proporcional a $1/N^2$. Desde $N \approx 10^6$ el error deja de bajar y hasta sube ($6{,}3 \cdot 10^{-13}$ con $N = 10^8$). Ahí manda el redondeo del tipo double (unos 16 dígitos significativos): cada uno de los N términos aporta un pequeño error de redondeo, y con más términos se acumula más. Es el mismo fenómeno de la sección 6.4. Con 8 hilos el error final es menor ($2{,}2 \cdot 10^{-14}$ con $N = 10^8$, frente a $6{,}3 \cdot 10^{-13}$ con 1 hilo; también se ve en la Figura 23), porque cada hilo acumula una suma parcial más pequeña y el error de redondeo crece menos.